

Reviewer Pack — Minimal K-theory Index and Communication Complexity Rank Ine...

Entry 1952fd32782b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 17:20:04 UTC

Minimal K-theory Index and Communication Complexity Rank Inequality

Entry ID: 1952fd32782b **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 17:20:04 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (K-theory) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For every n -communication protocol P , the minimal index of its associated vector bundle E is linearly correlated with its communication complexity rank $r(P)$, such that $\text{index}(E) = \Omega(r(P))$ and $\text{index}(E) \leq c * r(P)$ for some constant c .

Rationale (proposer's reasoning):

K-theory provides a comprehensive algebraic invariant capturing the topological structure of vector bundles, which may reflect the complexity of encoding information in communication protocols. A strong correlation between K-theoretic invariants and communication complexity could reveal deep connections between topology and computation.

Taxonomy category: K_theory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2e15c7be8c2404d7

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For an n -communication protocol P , if the computed minimal index of its associated vector bundle E is greater than or equal to 1.5 times the communication complexity rank $r(P)$ and less than or equal to 2 times $r(P)$, then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "K-theory index" AND "communication complexity rank" IN TITLE - "vector bundle" AND communication complexity AND K-theory - "algebraic topology" AND "matrix rank" AND protocol

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_protocol(n):
        # Generate a random n-communication protocol with known communication complexity rank r(P)
        return [random.randint(1, 40) for _ in range(n)]

    def construct_vector_bundle(protocol):
        # Construct the associated vector bundle E and compute its minimal index
        n = len(protocol)
        E = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(i, n):
                E[i][j] = protocol[j]
                E[j][i] = protocol[i]
        return E

    def min_index(E):
        # Compute the minimal index of the vector bundle E
        n = len(E)
        count = 0
        for i in range(n):
            if all(E[i][j] == 0 for j in range(i + 1, n)):
                count += 1
        return count

    def communication_complexity_rank(protocol):
        # Compute the communication complexity rank r(P) of the protocol
        return sum(protocol)
```

```

n = random.randint(5, 40)
protocol = generate_protocol(n)
E = construct_vector_bundle(protocol)
index_E = min_index(E)
r_P = communication_complexity_rank(protocol)

metric_name = 'minimal_index'
metric_value = index_E
instances_tested = 1
n_max = n
conjecture_holds = (1.5 <= index_E / r_P <= 2)
counterexample = f'Protocol with n={n}, index(E)={index_E}, r(P)={r_P}' if not conjecture_holds

return {
    'metric_name': metric_name,
    'metric_value': metric_value,
    'instances_tested': instances_tested,
    'n_max': n_max,
    'conjecture_holds': conjecture_holds,
    'counterexample': counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [random.randint(2, 997) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f'TRIAL: {result}')
        results.append(result)

    mean_value = sum(result['metric_value'] for result in results) / len(results)
    std_value = math.sqrt(sum((result['metric_value'] - mean_value) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result['conjecture_holds']) / len(results)

    if all(result['conjecture_holds'] for result in results):
        print(f'RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}')
    elif any(not result['conjecture_holds'] for result in results):
        first_failing_seed = next((result["seed"] for result in results if not result["conjecture_holds"]))
        print(f'RESULT: FALSIFIED counterexample="{results[first_failing_seed]["counterexample"]}"')
    else:
        print('RESULT: INCONCLUSIVE reason=unknown')

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

Protocol with n=7, index(E)=1, r(P)=142'}
TRIAL: {'metric_name': 'minimal_index', 'metric_value': 1, 'instances_tested': 1, 'n_max': 25, 'conjecture_holds': True}
TRIAL: {'metric_name': 'minimal_index', 'metric_value': 1, 'instances_tested': 1, 'n_max': 23, 'conjecture_holds': True}
TRIAL: {'metric_name': 'minimal_index', 'metric_value': 1, 'instances_tested': 1, 'n_max': 9, 'conjecture_holds': True}
TRIAL: {'metric_name': 'minimal_index', 'metric_value': 1, 'instances_tested': 1, 'n_max': 38, 'conjecture_holds': True}
TRIAL: {'metric_name': 'minimal_index', 'metric_value': 1, 'instances_tested': 1, 'n_max': 16, 'conjecture_holds': True}

```


12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1952fd32782b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/1952fd32782b.tar.gz` (if generated)